

Big Data — ELK & Grafana

Elasticsearch • Logstash • Grafana

Indexer, rechercher et visualiser les données anime à grande échelle.

AniData Lab — Séance 3 — Mardi 24 mars 2026 — Après-midi

Objectifs de cette séance

Ce que vous saurez faire à la fin de cet après-midi.

1

Comprendre la stack ELK

Architecture, rôles de chaque composant, cas d'usage.

2

Déployer avec Docker

Lancer Elasticsearch + Grafana en une commande.

3

Indexer le dataset gold

Charger anime_gold.json dans Elasticsearch.

4

Requêter les données

Recherche full-text, filtres, agrégations.

5

Créer des dashboards Grafana

Visualiser les tendances anime en temps réel.



CHAPITRE 1

C'est quoi la stack ELK ?

Trois outils open source qui forment le trio le plus utilisé pour le traitement de données à grande échelle.

ELK = Elasticsearch + Logstash + Kibana

Créé par Elastic, utilisé par Netflix, Uber, Wikipedia, la NASA...

Elasticsearch

**Moteur de recherche
et d'analyse**

Stocke les données, les indexe et permet de les rechercher en millisecondes. C'est le cœur de la stack.



Logstash

**Pipeline d'ingestion
de données**

Collecte les données depuis différentes sources, les transforme et les envoie vers Elasticsearch.



Grafana

**Visualisation
et dashboards**

Crée des dashboards interactifs pour explorer et monitorer les données en temps réel. Remplace Kibana dans notre stack.

Pourquoi utiliser ELK / Elastic Stack ?

Recherche ultra-rapide

Elasticsearch peut chercher dans des milliards de documents en millisecondes. Google utilise le même principe (index inversé).

Scalable

Passe de quelques Go à des Pétaoctets. Ajoutez des nœuds, la capacité augmente. Netflix gère 150 milliards de logs/jour.

Temps réel

Les données sont disponibles en moins d'une seconde après l'indexation. Idéal pour le monitoring et les alertes.

Open Source

Gratuit et communautaire. Utilisé par des milliers d'entreprises. Compétence très demandée sur le marché.

Cas d'usage : monitoring applicatif, analyse de logs, recherche full-text, analytics business, sécurité (SIEM), observabilité.

Kibana vs Grafana : pourquoi Grafana ?

Dans notre stack, on utilise Grafana à la place de Kibana. Voici pourquoi.

Critère	Kibana	Grafana
RAM	~512 Mo	~128 Mo ✓
Sources	Elasticsearch seul	Multi-sources (ES, SQL, Prometheus...) ✓
Prise en main	Complexe	Plus intuitive ✓
Recherche full-text	Excellente ✓	Basique
Marché de l'emploi	Stack ELK classique	DevOps / SRE / Data

Pour notre projet (8 Go RAM + débutants), Grafana est 4× plus léger et plus simple. On garde Elasticsearch comme moteur — seule la couche de visualisation change.



CHAPITRE 2

Elasticsearch en profondeur

Le moteur de recherche et d'analyse qui stocke et indexe vos données.

Les concepts clés d'Elasticsearch

Elasticsearch organise les données différemment d'une base SQL classique.

Index = une « base de données »

Contient tous les documents d'un même type. Ex : index « anime ».

Document = une « ligne »

Un objet JSON stocké dans un index. Ex : un anime avec tous ses champs.

Mapping = un « schéma »

Définit les types des champs (text, keyword, integer, date...).

Shard = une « partition »

Un index est découpé en shards pour la performance et la scalabilité.

SQL vs Elasticsearch : le vocabulaire

SQL	Elasticsearch	AniData Lab
Base de données	Cluster	anidata-cluster
Table	Index	anime
Ligne (row)	Document (JSON)	{ name: "Naruto", score: 8.0 }
Colonne	Field (champ)	name, score, genres...
Schéma	Mapping	score: float, name: text
SELECT * WHERE...	Query DSL (JSON)	{ match: { name: "Naruto" } }

La différence fondamentale : Elasticsearch est orienté recherche (index inversé), pas orienté transactions. Il est optimisé pour la lecture rapide, pas pour les updates fréquents.

Le secret d'Elasticsearch : l'index inversé

C'est ce qui rend la recherche ultra-rapide, même sur des milliards de documents.

Approche classique (SQL)

Parcourir CHAQUE document
pour trouver le mot « ninja »

Doc 1 : « Naruto est un ninja... » →

✓

Doc 2 : « One Piece pirate... » → ✗

Doc 3 : « Ninja Scroll action... » →

✓

...

Doc 17562 : ...

→ Lent ! $O(n)$ documents

Index inversé (Elasticsearch)

Table pré-calculée :

« ninja » → [Doc 1, Doc 3, Doc 89]

« pirate » → [Doc 2, Doc 45]

« mecha » → [Doc 7, Doc 12, Doc 99]

« magic » → [Doc 4, Doc 8, Doc 56]

Rechercher « ninja » = juste
lire la liste ! $O(1)$

→ Instantané !

Le mapping : définir la structure des données

Le mapping dit à Elasticsearch comment interpréter chaque champ.

text

Texte analysé et découpé en mots (tokenisé). Pour la recherche full-text. Ex : synopsis.

keyword

Texte exact, non analysé. Pour les filtres et agrégations. Ex : genre, studio.

integer / float

Nombres entiers ou décimaux. Pour les calculs et les ranges. Ex : score, episodes.

date

Dates au format ISO. Pour les filtres temporels. Ex : aired_start.

boolean

Vrai ou faux. Pour les flags. Ex : is_outlier.

nested / object

Objets imbriqués ou listes d'objets. Pour les structures complexes.

Notre mapping : index « anime »

```
PUT /anime
```

```
{  
  "mappings": {  
    "properties": {  
      "name": { "type": "text" },  
      "japanese_name": { "type": "text" },  
      "score": { "type": "float" },  
      "weighted_score": { "type": "float" },  
      "genres": { "type": "keyword" },  
      "main_genre": { "type": "keyword" },  
      "main_studio": { "type": "keyword" },  
      "studio_tier": { "type": "keyword" },  
      "type": { "type": "keyword" },  
      "episodes": { "type": "integer" },  
      "members": { "type": "long" },  
      "drop_ratio": { "type": "float" },  
      "decade": { "type": "integer" },  
      "score_category": { "type": "keyword" }  
    }  
  }  
}
```

Mapping adapté au dataset
anime_gold.json.

Recherche dans Elasticsearch (Query DSL)

Elasticsearch utilise des requêtes en JSON au lieu de SQL.

```
// Recherche full-text
GET /anime/_search
{ "query": {
  "match": { "name": "naruto" }
}}
```

Recherche full-text

```
// Filtre par plage
GET /anime/_search
{ "query": { "range": {
  "score": {
    "gte": 8,
    "lte": 10
  }
}
}}
```

Range (entre 8 et 10)

```
// Filtre exact
GET /anime/_search
{ "query": { "term": {
  "main_genre": "Action" }
}}
```

Filtre exact (keyword)

```
// Agrégation (top genres)
GET /anime/_search
{ "size": 0,
  "aggs": {
    "top_genres": {
      "terms": {
        "field": "main_genre" }
    }
  }
}
```

Agrégation (comptage)



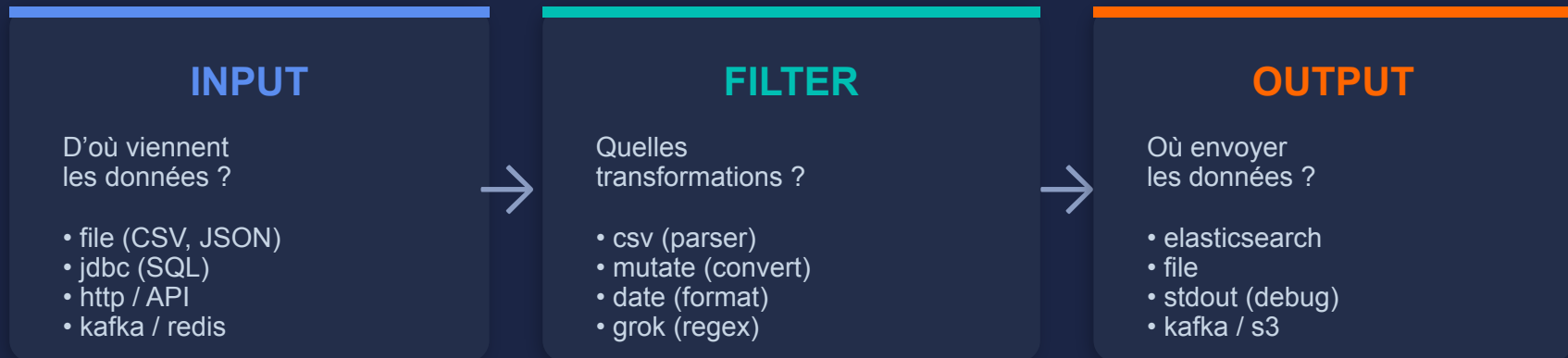
CHAPITRE 3

Logstash

Le pipeline d'ingestion qui transforme et charge les données dans Elasticsearch.

Logstash : Input → Filter → Output

Un pipeline Logstash se compose toujours de 3 sections.



Logstash est lancé à la demande dans notre stack (profil Docker « ingest »). Il lit le fichier, le transforme, l'envoie dans ES, puis s'arrête. Ce n'est pas un service permanent.

Notre pipeline Logstash : anime.conf

```
input {  
  file {  
    path => "/data_input/anime.csv"  
    start_position => "beginning"  
  }  
}  
  
filter {  
  csv { separator => "," columns => ["mal_id","name","score",...] }  
  mutate {  
    convert => { "score" => "float" "episodes" => "integer" }  
  }  
  if [genres] { mutate { split => { "genres" => ", " } } }  
}  
  
output {  
  elasticsearch {  
    hosts => ["http://elasticsearch:9200"]  
    index => "anime"  
    document_id => "%{mal_id}"  
  }  
}
```

Configuration utilisée pour indexer anime.csv dans Elasticsearch.

Alternative : indexer directement en Python

```
from elasticsearch import Elasticsearch, helpers
import json

es = Elasticsearch("http://localhost:9200")

# Charger le NDJSON
with open("output/anime_gold.json") as f:
    docs = [json.loads(line) for line in f]

# Indexation bulk (par paquets de 500)
actions = [
    { "_index": "anime", "_id": doc.get("mal_id"), "_source": doc }
    for doc in docs
]

helpers.bulk(es, actions, chunk_size=500)
print(f"✅ {len(docs)} documents indexés !")
```

Pour notre dataset gold (déjà nettoyé en JSON), on peut sauter Logstash.

Avantage : pas besoin de lancer Logstash (512 Mo de RAM économisés). Idéal pour notre setup 8 Go.



CHAPITRE 4

Grafana

Créer des dashboards interactifs pour explorer les données anime.

Les concepts clés de Grafana

Datasource

Connexion à une base de données. On connecte Grafana à Elasticsearch. Déjà pré-configurée dans notre docker-compose.

Dashboard

Page contenant plusieurs panels (graphiques). On peut en créer autant qu'on veut. Export/import en JSON.

Panel

Un graphique individuel : bar chart, pie chart, stat, table, heatmap... Chaque panel fait une requête à ES.

Query

La requête que Grafana envoie à Elasticsearch. On utilise les agrégations ES pour compter, moyenner, filtrer.

Types de panels Grafana

Choisir le bon type de panel selon ce qu'on veut montrer.

Stat

Un gros chiffre. Idéal pour : total animes, score moyen, nombre d'utilisateurs.

Bar chart

Barres horizontales ou verticales. Idéal pour : top studios, top genres, comparaisons.

Pie / Donut

Camembert. Idéal pour : répartition par type (TV, OVA, Movie), par tier de studio.

Table

Tableau de données brutes. Idéal pour : liste des top animes avec détails.

Time series

Courbe temporelle. Idéal pour : évolution du nombre d'animes par décennie.

Heatmap

Carte de chaleur. Idéal pour : corrélation score/popularité.

Dashboard pré-configuré : AniData Lab

Un dashboard de démarrage est déjà inclus dans le docker-compose.

4 panels prêts à l'emploi :

1. Total animes indexés

Panel Stat — affiche le compteur total de documents dans l'index.

2. Score moyen

Panel Stat — calcule la moyenne des scores avec agrégation ES avg.

3. Répartition par type

Panel Pie chart — répartition TV / OVA / Movie / Special via terms aggregation.

4. Top 10 studios

Panel Bar chart — comptage par studio via terms aggregation, tri descendant.



CHAPITRE 5

Mise en pratique

Indexer le dataset gold et créer vos dashboards.

Plan de l'après-midi (2h30 de pratique)

0:00 – 0:30

Vérifier que Docker tourne, ES répond sur :9200

0:30 – 1:00

Indexer anime_gold.json dans Elasticsearch (Python ou Logstash)

1:00 – 1:15

Tester des requêtes dans le terminal (curl)

1:15 – 2:00

Créer 3-4 panels Grafana supplémentaires

2:00 – 2:30

Exploration libre + questions

Étape 1 : Vérifier l'environnement

```
# Vérifier que tout tourne
docker compose ps

# Vérifier Elasticsearch
curl http://localhost:9200
# → Doit retourner un JSON avec le nom du cluster

# Vérifier Grafana
# → Ouvrir http://localhost:3000 (admin / anidata)

# Si ES ne répond pas :
docker compose logs elasticsearch
# Sur Linux : sudo sysctl -w vm.max_map_count=262144
```

Étape 2 : Indexer le dataset gold

Deux options selon votre préférence.

Option A — Python (recommandé)

```
python scripts/06_indexation_es.py
```

Utilise la librairie elasticsearch-py
pour envoyer les documents en bulk.
Pas besoin de Logstash !

Option B — Logstash

```
docker compose \  
--profile ingest up logstash
```

Utilise le fichier anime.conf.
Plus lourd (512 Mo) mais c'est
l'approche « officielle » ELK.

```
# Vérifier que l'indexation a fonctionné  
curl http://localhost:9200/anime/_count  
# → { "count": 17562, ... }
```

```
# Chercher un anime  
curl -X GET "http://localhost:9200/anime/_search?q=name:naruto&pretty"
```

Étape 3 : Tester des requêtes

Quelques requêtes à essayer dans le terminal.

```
# Top 5 animes par score
curl -s localhost:9200/anime\
/_search -H "Content-Type:\
application/json" -d '{
  "size": 5,
  "sort": [{"score": "desc"}]
}' | python -m json.tool
```

Top 5 par score

```
# Compter par genre
curl -s localhost:9200/anime\
/_search -H "Content-Type:\
application/json" -d '{
  "size": 0,
  "aggs": {"genres":
    {"terms": {"field": "main_genre"}}}
}' | python -m json.tool
```

Comptage par genre

```
# Recherche full-text dans les noms
curl -s "localhost:9200/anime/_search?q=name:dragon+ball&pretty"

# Score moyen par décennie
curl -s localhost:9200/anime/_search -H "Content-Type: application/json" \
  -d '{"size":0,"aggs":{"decades":{"terms":{"field":"decade"},"aggs":{"avg_score":{"avg":
{"field":"score"}}}}}}'
```

Étape 4 : Créer un panel Grafana

Procédure pas à pas pour ajouter un panel au dashboard.

1

Ouvrir le dashboard

<http://localhost:3000> → Dashboards → AniData Lab → Edit

2

Add panel

Cliquer sur « Add » → « Visualization » en haut à droite.

3

Choisir la datasource

Sélectionner « Elasticsearch - Anime » (déjà configurée).

4

Configurer la requête

Metric = Count ou Average(score). Group by = terms(main_genre).

5

Choisir la visualisation

Bar chart, Pie chart, Table, Stat... selon le besoin.

6

Sauvegarder

Donner un titre au panel, cliquer « Apply » puis « Save dashboard ».

Idées de panels à créer

En plus des 4 panels pré-configurés, voici des idées pour aller plus loin.

Score moyen par décennie

Time series ou Bar chart — Metric: avg(score), Group by: terms(decade)

Top 10 animes les plus populaires

Table — Sort by: members desc, Size: 10, Columns: name, score, members

Répartition par tier de studio

Pie chart — Metric: count, Group by: terms(studio_tier)

Distribution des scores

Histogram — Metric: count, Group by: histogram(score, interval=0.5)

Ratio d'abandon par genre

Bar chart — Metric: avg(drop_ratio), Group by: terms(main_genre)

Animes par catégorie de score

Pie chart — Metric: count, Group by: terms(score_category)

Ce qu'il faut retenir

Elasticsearch stocke et indexe les données pour une recherche ultra-rapide (index inversé).

Un index = une base, un document = une ligne JSON, un mapping = un schéma.

Logstash est un pipeline Input → Filter → Output pour l'ingestion de données.

On peut aussi indexer directement en Python avec elasticsearch-py (plus léger).

Grafana crée des dashboards interactifs connectés à Elasticsearch.

Query DSL : match (full-text), term (exact), range (plage), aggs (agrégations).

Le dataset anime_gold.json est prêt à être indexé tel quel (NDJSON).

Demain : on automatise tout ça avec Apache Airflow !

Et maintenant ?

Demain : Apache Airflow !

Ce que vous avez accompli aujourd'hui :

- Nettoyé et enrichi le dataset anime (8 features métier)
- Indexé 17 562 animes dans Elasticsearch
- Créé un dashboard Grafana interactif

Demain, on automatise tout ce pipeline avec Apache Airflow pour qu'il s'exécute en un clic.